Print copy of official Microsoft Learn documentation

Source: https://learn.microsoft.com/en-us/dotnet/core/tools/
Exported: 2026-10-14. Source metadata date: 2026-09-29T00:00:00Z.

Microsoft and documentation contributors. Article wording and code for the requested version are retained; web navigation is removed and print formatting is added. This locally generated PDF is not a Microsoft-published PDF book or a complete product manual. Source-page attribution and notices apply.

Documentation license and terms: Microsoft Learn terms; .NET documentation: CC BY 4.0 license. Check the source for specific attribution or exceptions.

.NET CLI overview

This article applies to: ✔ .NET Core 3.1 SDK and later versions

The .NET command-line interface (CLI) is a cross-platform toolchain for developing, building, running, and publishing .NET applications.

The .NET CLI is included with the .NET SDK. For more information about how to install the .NET SDK, see Install .NET. After installing the SDK, you can run CLI commands by opening a terminal and entering the commands at the terminal prompt.

Command structure

CLI command structure consists of the driver ("dotnet"), the command, and possibly command arguments and options. You see this pattern in most CLI operations, such as creating a new console app, and running it from the command line. The following commands show when the console app was run from a directory named *my_app*:

```
dotnet new console
dotnet build --output ./build_output
dotnet ./build_output/my_app.dll
```

Driver

The driver is named dotnet and has two responsibilities, either running a framework-dependent app or executing a command.

To run a framework-dependent app, specify the path to the app's .dll file after the driver without specifying a command, for example, `dotnet /path/to/my_app.dll`. When executing the command from the folder where the app's DLL resides, just execute `dotnet my_app.dll`. For more information, see the dotnet command.

When you supply a command to the driver, `dotnet.exe` starts the CLI command execution process. For example:

```
dotnet build
```

First, the driver determines the version of the SDK to use. If there's no global.json file, the latest version of the SDK available is used. After the SDK version is determined, it executes the command.

Command

The command performs an action. For example, `dotnet build` builds code. `dotnet publish` publishes code. See the CLI commands section for a list of commands.

Arguments

The arguments you pass on the command line are the arguments to the command invoked or to options specified with the command. For example, when you execute `dotnet publish my_app.csproj`, the `my_app.csproj` argument indicates the project to publish and is passed to the `publish` command.

Options

The options you pass on the command line are the options to the command invoked. For example, when you execute `dotnet publish --output /build_output`, the `--output` option and its value provided by the `/build_output` argument are passed to the `publish` command.

CLI commands

The following commands are installed by default:

Basic commands

- new
- restore
- build
- publish
- run
- test
- vstest
- pack
- clean
- sln
- help
- store
- watch
- format (Available since .NET 6 SDK)

Project modification commands

- package add
- package download
- package list

- package remove
- package search
- package update
- project convert (Available since .NET 10 SDK)
- reference add
- reference list
- reference remove

NuGet commands

- nuget delete
- nuget locals
- nuget push
- nuget add source (Available since .NET Core 3.1.200 SDK)
- nuget disable source (Available since .NET Core 3.1.200 SDK)
- nuget enable source (Available since .NET Core 3.1.200 SDK)
- nuget list source (Available since .NET Core 3.1.200 SDK)
- nuget remove source (Available since .NET Core 3.1.200 SDK)
- nuget update source (Available since .NET Core 3.1.200 SDK)
- nuget verify (Available since .NET 5 SDK)
- nuget trust (Available since .NET 5 SDK)
- nuget sign (Available since .NET 6 SDK)
- nuget why (Available since .NET 8.0.4xx SDK)

Workload management commands

- workload (Available since .NET 7 SDK)
- workload clean (Available since .NET 6 SDK)
- workload config (Available since .NET 8.0.400 SDK)
- workload install (Available since .NET 6 SDK)
- workload history (Available since .NET 6 SDK)
- workload list (Available since .NET 6 SDK)
- workload update (Available since .NET 6 SDK)
- workload restore (Available since .NET 6 SDK)
- workload repair (Available since .NET 6 SDK)
- workload uninstall (Available since .NET 6 SDK)
- workload search (Available since .NET 6 SDK)

Advanced commands

- sdk check
- msbuild
- build-server
- dev-certs
- dotnet install script

Tool management commands

- `tool install`
- `tool list`
- `tool update`
- `tool restore`
- `tool run`
- `tool uninstall`
- `tool search`

Tools are console applications that are installed from NuGet packages and are invoked from the command prompt. You can write tools yourself or install tools written by third parties. Tools are also known as global tools, tool-path tools, and local tools. For more information, see .NET tools overview.

Manage SDK installations with dotnetup

Use dotnetup to manage user-level .NET SDK and runtime installations, configure access to those installations, and track version requirements.

See also

- dotnet/sdk GitHub repository
- .NET installation guide